



# **Fidessa Product Infrastructure**

**Version 8.0**

## **OpenAccess Stream Server Configuration Guide**

**Issue: 1.2**

**17th April 2012**

**Fidessa**  
Trade/Invest/Inform

## Change History

| Issue Date                  | Issue Number | Description of Changes                                                                                                                                                       |
|-----------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 <sup>rd</sup> May 2011    | 1.0          | Updated for infrastructure v8.0.0: added a note on the requirement for the primary table to be configured with a guaranteed tick update queue. See section 2.10 for details. |
| 18 <sup>th</sup> April 2012 | 1.1          | Corrected the description of the username field in section 2.7.                                                                                                              |
| 17 <sup>th</sup> April 2014 | 1.2          | Updated for infrastructure v8.0.12: added details of the fixedFields configuration block.                                                                                    |

© 2025 **Fidessa**.

All rights reserved. This document may be used only for the purpose for which it has been provided. In particular, the contents may not be reproduced or transmitted in whole or in part, in any form or by any means without the express written permission of **Fidessa**. Any and all **Fidessa** software described herein and any associated documentation (including this document) is furnished under a separate licence agreement and its use (including any copying) is strictly subject to the terms of that agreement.

Fidessa ® is a registered trademark of **Fidessa** group plc.

# Contents

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| <b>1.</b> | <b>Introduction .....</b>                             | <b>5</b>  |
| 1.1       | General.....                                          | 5         |
| 1.2       | RDS.....                                              | 5         |
| 1.3       | Transaction Streams .....                             | 6         |
| 1.4       | Related Documents.....                                | 7         |
| <b>2.</b> | <b>Configuration Notes.....</b>                       | <b>9</b>  |
| 2.1       | general Block .....                                   | 9         |
| 2.2       | expression Block .....                                | 9         |
| 2.3       | linkMgr Block.....                                    | 10        |
| 2.4       | streamNode Block.....                                 | 10        |
| 2.5       | Configuring RDS .....                                 | 12        |
| 2.6       | Customising RDS.....                                  | 26        |
| 2.7       | Configuring Transaction Streams .....                 | 27        |
| 2.8       | Customising Transaction Streams .....                 | 35        |
| 2.9       | Data Fresh Where .....                                | 36        |
| 2.10      | Special Note: Guaranteed Ticks .....                  | 38        |
| <b>3.</b> | <b>Process operation.....</b>                         | <b>39</b> |
| 3.1       | Command Line Options .....                            | 39        |
| <b>4.</b> | <b>Sample Configuration.....</b>                      | <b>40</b> |
| <b>5.</b> | <b>Appendix A: The <code>EMBED XLator</code>.....</b> | <b>44</b> |
| <b>6.</b> | <b>Appendix B: OpenAccess Statistics.....</b>         | <b>47</b> |
| 6.1       | Class <code>&lt;SERVICE_NAME&gt;.RDS</code> .....     | 47        |
| 6.2       | Class <code>&lt;SERVICE_NAME&gt;.TXN</code> .....     | 49        |
| 6.3       | Class <code>&lt;SERVICE_NAME&gt;.INFO</code> .....    | 49        |

7.     **Appendix C: Pitfalls ..... 51**

      7.1     Global Variable Use ..... 51

      7.2     Backward Joins ..... 51

      7.3     Update Accumulation ..... 52

      7.4     Stream Resumption ..... 52

8.     **Appendix D: The SOURCE\_KEY field..... 53**

      8.1     Backward Compatibility..... 53

# 1. Introduction

## 1.1 General

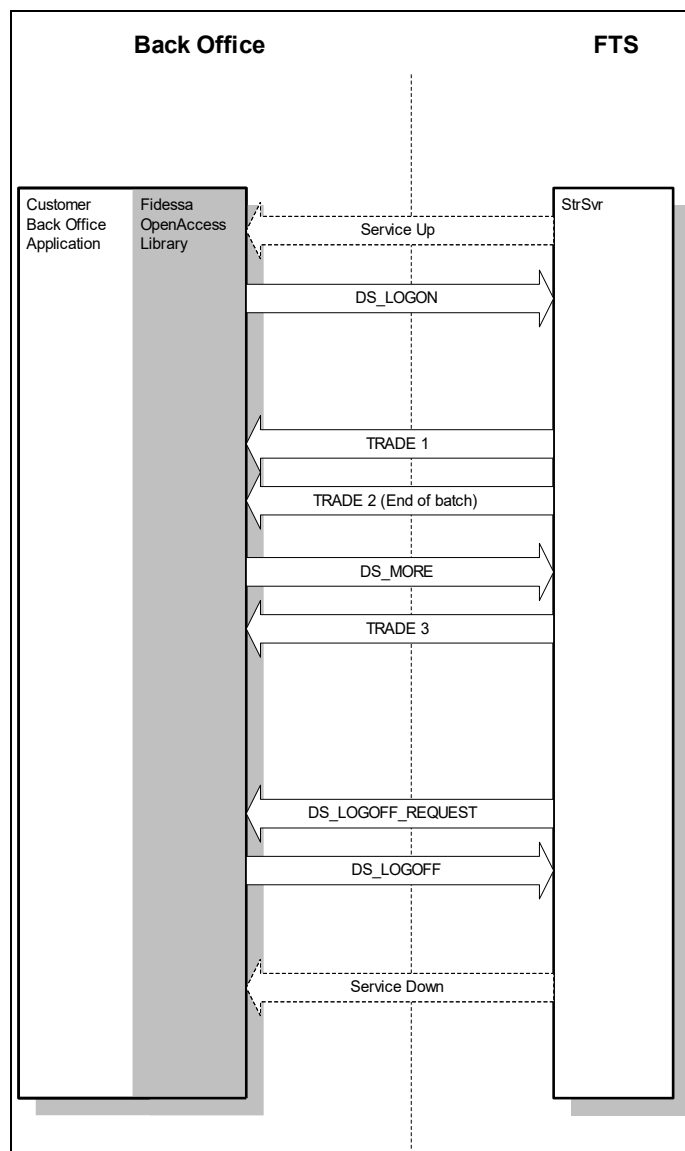
The Stream Server (`StrSvr`) is a generic infrastructure component which can be used to provide streams of data over OpenAccess conforming to either the Reliable DataStream (RDS) or Transaction protocol. When providing an RDS feed, `StrSvr` acts as a server, accepting incoming connections from a client and supplying data depending on requests sent by that client. When providing a transaction stream, `StrSvr` connects to a transaction server and sends in transactions as defined by the configuration. Both streams can be recovered or resumed in the manner defined by the OpenAccess Programmer's Guide.

The following summarises the key features of each stream as implemented by `StrSvr`.

## 1.2 RDS

- A resumable stream of data, generated by scanning a configured combination of relationally joined tables.
- Provided in batches.
- Translation of database data in SDS format can be done either through configuration or through bespoke code, linked against the stream server library (`libFIDStrNode`).
- Real-time updates supported
- Optionally, user access can be restricted on a per-datastream basis.

The following diagram shows a possible context for `StrSvr` when providing an RDS stream. Here, `StrSvr` is forming the interface between the Fidessa Trading System, and a customer back office application.

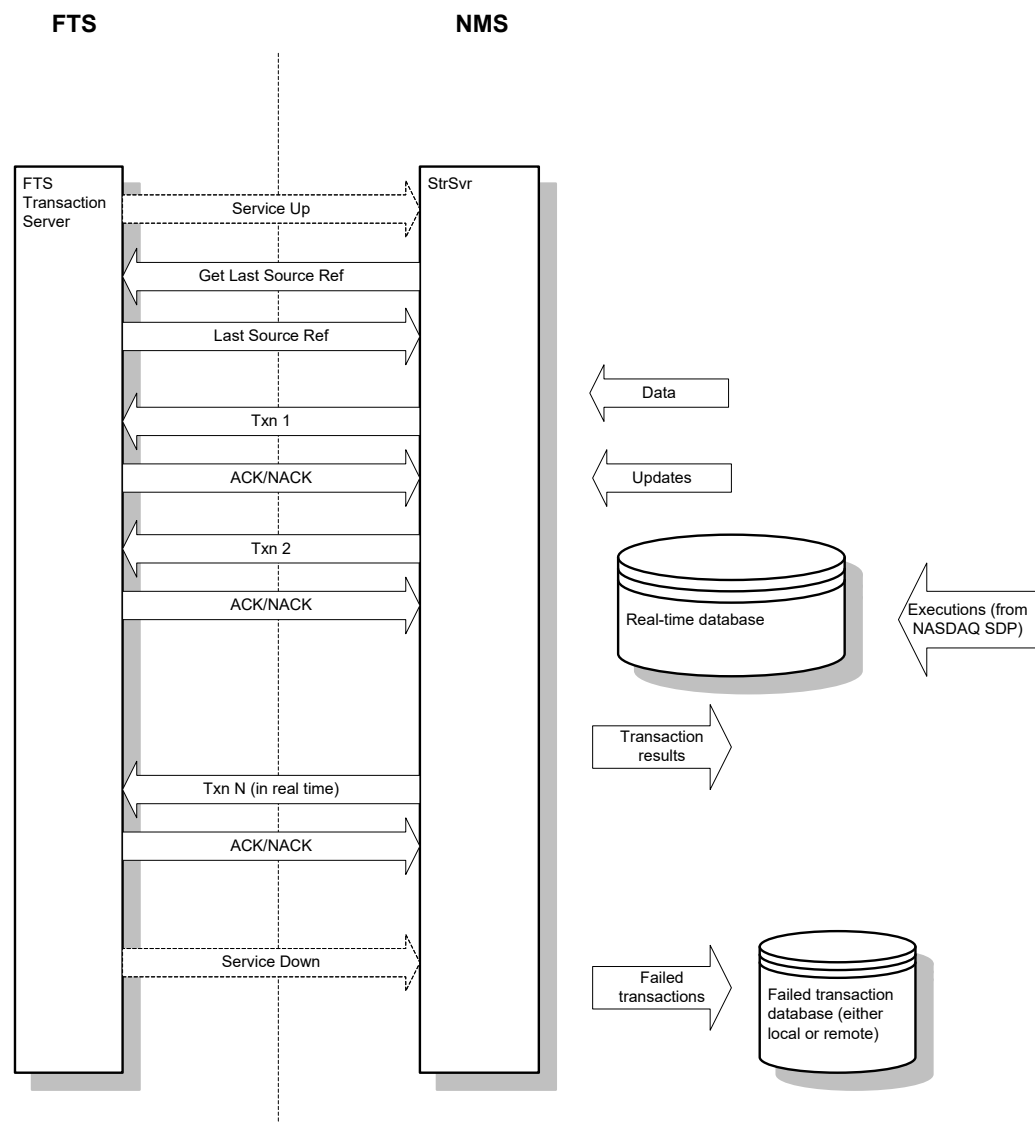


### 1.3 Transaction Streams

- Resumable transaction stream with recoverable statuses. The stream is resumed at the point at which it was curtailed by requesting the last processed source reference from the server. Statuses of a number of preceding transactions are also recovered from the server.
- Generated by scanning a configured combination of relationally joined tables.
- Synchronous or asynchronous transaction transmission
- Translation of database data in SDS format can be done either through configuration or through bespoke code, linked against the stream server library.
- Real-time updates supported

- Transaction results can either simply be written to a local database table or played back into either a local or remote system via a transaction. The latter allows transaction statuses to be present on both machines of a live/standby pair.

The following diagram shows `StrSvr` providing a guaranteed stream of transactions from one system to another. Here, executions from the NASDAQ exchange are being notified to FTS (from the Fidessa NASDAQ Management System) via a stream of transactions.



## 1.4 Related Documents

For an introduction to the OpenAccess suite of protocols, and specific details of the fields and message constructs supported by the various systems, please refer to the following documents.

| Number | Title                                                                      |
|--------|----------------------------------------------------------------------------|
| 1      | Fidessa OpenAccess: Programmers Guide for v1.2                             |
| 2      | Fidessa FTS : OpenAccess : Message Specification                           |
| 3      | Fidessa OMAR : OpenAccess : Message Specification                          |
| 4      | Fidessa NMS : OpenAccess : Message Specification                           |
| 5      | Fidessa Product Infrastructure : Expression Evaluator : Programmer's Guide |



## 2. Configuration Notes

This section takes you through the items which can be used to modify the behaviour of `StrSvr` through configuration.

### 2.1 general Block

The outer block of the configuration file should be called `StrSvr`. Inside here, the first thing that should appear is a block called `general` - the top of an example configuration might look like this:

```
StrSvr
{
    general
    {
        processName          STRSVR
        streamServiceName    FTS_RDS
    }
}
```

The `general` block contains two items, both of which are optional. These are described in the following table.

| Name                           | Default | Description                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>processName</code>       | STRSVR  | Defines the name used when the process logs a message in the system-wide operator log. You should make this name unique within your system.                                                                                                                                                                                                 |
| <code>streamServiceName</code> | STREAM  | Name of the OpenAccess service which this application will provide. When the application starts, the OpenAccess library will check in the <code>OAServices</code> file for this name, and if it is present, it will declare itself on the specified port. Clients will then be able to connect to this service to request your RDS streams. |

Note that the `general` block can be omitted, and if it is, the items within it will take the default values mentioned above.

### 2.2 expression Block

Following the `general` block, the standard expression block may optionally be present:

```

expression
{
    dll ( "libIDSFTS.so" )

    xrefTables
    {
        Roman
        {
            I    One
            II   Two
            III  Three
            IV   Four
            V    Five
        }
    }

    preExpression ( "Display('Evaluating StrSvr pre-expressions\n')" )
}

```

The `expression` block allows you to configure attributes of the expression evaluator, including the following:

- dynamically linked expression libraries (libraries provided by royalblue which provide business-specific expression functions)
- cross-reference tables (which install cross-referencing functions for use in subsequent expressions)
- pre-expressions (a list of expressions which are evaluated once, at start-up)

For full details, see the Fidessa Expression Evaluator Programmer's Guide.

## 2.3 linkMgr Block

After this block, the OpenAccess library is configured, using the `linkMgr` block. For more details concerning this block refer to the Expression Evaluator Programmer's Guide.

## 2.4 streamNode Block

Following this general configuration is the `streamNode` block, which defines the configuration of each stream. The top of a `streamNode` block might look like this:

```

streamNode
{
    general
    {
        upqPollPeriod    1000
        asyncBatchDelay  10
        asyncBatchSize   100
        delimiterChar     ^
    }
}

```

```

        rdsLogoffTimeout 5000
    }

```

The `general` block specifies several parameters which affect all templates, as described below.

| Name                          | Default | Description                                                                                                                                                                                                                                                               |
|-------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>upqPollPeriod</code>    | 500     | Determines how often the update queue is polled for updates, in milliseconds.                                                                                                                                                                                             |
| <code>asyncBatchSize</code>   | 100     | Scans for matching rows are timesliced in the stream server. This configuration option specifies the number of rows which will be scanned before queuing a timer and returning to the event loop.                                                                         |
| <code>asyncBatchDelay</code>  | 10      | Length of timer, in milliseconds, queued between batches of database scanning.                                                                                                                                                                                            |
| <code>delimiterChar</code>    | (pipe)  | Specifies the character used to delimit the strings which make up datastream reference and source reference. <b>Not normally specified.</b>                                                                                                                               |
| <code>rdsLogoffTimeout</code> | 30000   | Timeout, in milliseconds, that <code>StrSvr</code> waits after sending out a logoff request before forcibly logging the client off.                                                                                                                                       |
| <code>maxLogons</code>        | -1      | Defines the maximum number of permitted logons to each RDS Template defined within the <code>streamNode</code> block. The default (-1) indicates that any number of logons is permitted. The value specified within this block may be overridden on a per-template basis. |

### 2.4.1 Delimiters and Stream References

The `delimiterChar` item specifies the character used to delimit the strings which make up the datastream reference and source reference - the unique identifiers which are supplied with each message in the respective stream protocols. Logically, references are built from database key values (as these are unique to each record) as the concatenation of the string conversions of the fields in a key. The fields are delimited by the delimiter character, so that when recovering a stream, `StrSvr` can quickly reconstruct the key value corresponding to a reference. Hence the delimiter character should be guaranteed not to appear in your string data. By default, `StrSvr` uses the pipe character (|), but if this is used in your string data, you will need to use an alternative.

**Note:** The pipe character was chosen because it rarely appears in string data. You are strongly advised to use this default.

### 2.4.2 Logoff Timeout

At the end of a day, when the system on which `StrSvr` is running is shutting down, `StrSvr` sends out RDS logoff request messages to all connected clients, and waits for them to respond with a logoff. If a client does not respond within a given timeout, the session is aborted. You can specify this timeout, in milliseconds, in the `rdsLogoffTimeout` item.

Following this, you define the RDS and transaction streams you want to supply in a series of blocks called respectively `rdsTemplate` and `txnTemplate`.

## 2.5 Configuring RDS

The `rdsTemplate` block has the following format:

```
rdsTemplate
{
    name          TRADE_STREAM
    key           TRADE_SUMMARY_BY_SET_SORT_VERSION
    defaultMaxCount 100
    rowsPerTimeslice 100
    reverseSort    NO
    XLator        NONE
    maxUpdatesQueued 100000

    accessCheck
    {
        rightsType FTS_RDS
        rightsCode TRADE_STREAM_VIEW
    }

    onLogon
    {
        tableName    LOG_EVENTS
        alias         le
        keyName       LOG_EVENTS_BY_USER_OA_RDS

        where         "userName=='REQD_USER'"

        fieldExprs ( "le.USER_NAME=userName",
                    "le.OA_SERVICE=oaService",
                    "le.RDS_NAME=rdsName",
                    "le.LOG_STATUS=logonFlag" )
    }

    table
    {
        name TRADE_SUMMARY
        alias ts
    }
}
```

```

        ! Illustrative "where" clause picks out only
        ! principal trades
        where "ts.TRADE_BASIS=='PR'"
    }
    table
    {
        name BOOK
        alias bk

        ! Note the single = in the join expression
        ! The join *copies* fields from the primary record to
        ! this one - these fields are used to "seed" a
        ! database lookup
        join "bk.BOOK_ID = ts.BOOK_ID"
        backwardJoin "ts.BOOK_ID = bk.BOOK_ID"
    }

    fieldExprs ( "TRADE_SET_ID.s = ts.TRADE_SET_ID" )
}

```

This block defines everything about the RDS stream, as summarised below.

| Name             | Default                    | Description                                                                                                                                                                                                                       |
|------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| name             | None<br><b>(Mandatory)</b> | Datastream name which the client supplies at logon time.                                                                                                                                                                          |
| key              | None<br><b>(Mandatory)</b> | Sorted database key on the primary table to use for scanning and for generating datastream references.                                                                                                                            |
| reverseSort      | NO                         | Set to YES if the specified key is monotone decreasing with time, or NO if the key is monotone increasing.                                                                                                                        |
| defaultMaxCount  | 100                        | Default number of rows in a batch.                                                                                                                                                                                                |
| maxLogons        | -1                         | Maximum number of permitted logons to the RDS Stream. If not specified, the default (-1) indicates that any number of logons are permitted. This value overrides any 'maxLogon' settings specified within the 'streamNode' block. |
| rowsPerTimeslice | 100                        | Number of rows read before "timeslicing", or moving on to service another client.                                                                                                                                                 |

**New in  
v6.1.16**

| Name                                     | Default            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>XLator</code>                      | DEFAULT            | Defines translator between database records and datastream messages. Can be DEFAULT, EMBED or NONE. Other values may be valid in applications linked against the stream server library – see below for further details.                                                                                                                                                                                                                                                                                                    |
| <code>maxUpdatesQueued</code>            | 100000             | The maximum number of updates that may be queued on a datastream session before it is aborted. This is to prevent arbitrarily large numbers of updates being buffered if a client stops responding.                                                                                                                                                                                                                                                                                                                        |
| <code>accessCheck</code>                 | No access checking | <p>This optional block defines access checking parameters. User access to datastreams can be restricted on a per-datastream basis.</p> <p>If the block is omitted, users' access to datastreams is not checked. If it is present, users are denied access to datastreams if either they are not valid users of the system, or if they do not have the correct RIGHTS_TYPE/RIGHTS_CODE combination.</p> <p>If the block is present but empty, rights are checked, and the default RIGHTS_TYPE and RIGHTS_CODE are used.</p> |
| <code>accessCheck/<br/>rightsType</code> | OA_RDS             | Defines the RIGHTS_TYPE to look for when performing an access check.                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>accessCheck/<br/>rightsCode</code> | Datastream name    | Defines the RIGHTS_CODE to look for when performing an access check.                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

**Updated  
in v6.1.0**

| Name                  | Default                           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| onLogon blocks        | No recording of Log-on/off events | <p>Include these optional blocks to enable the recording of Log-on and Log-off events; omitting it disables the feature. Each time a <code>DS_LOGON</code> message is received, and the connection accepted, a number of records may be entered into the rtd. Similarly, when the client logs off, or the session is aborted, the records will be modified to show this. When recording an event in the database table, if a matching record can be found it will be modified; otherwise a new record will be created.</p> <p>Four embedded variables are made available to all the expressions that are defined within this block. The variables are named <code>userName</code>, <code>oaService</code>, <code>rdsName</code> and <code>logonFlag</code> and are all of string type. <code>userName</code> contains the user name of the client; <code>oaService</code> contains the service name of the Stream Server; <code>rdsName</code> contains the name of the rdsTemplate; <code>logonFlag</code> contains either "LOGGED_ON" or "LOGGED_OFF". Furthermore, the <code>logon</code> alias can be used from within these expressions; it refers to the logon message sds.</p> |
| onLogon/<br>tableName |                                   | Specifies the rtd table that is to receive the records of log-on and log-off events.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| onLogon/<br>alias     |                                   | Specifies the table alias to be used in the <code>onLogon/fieldExprs</code> strings (below).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| onLogon/<br>keyName   |                                   | Specifies the key to be used to find matching records in the rtd table.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

|                          | Name                   | Default                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------|------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | onLogon/<br>where      |                            | An integer-valued expression string that is evaluated for every log-on/off event. A record of the event is only made if the expression evaluates to a non-zero value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|                          | onLogon/<br>fieldExprs |                            | A list of expressions that will be evaluated when a log-on/off event is to be recorded. They should be used to populate the record that is to be entered in the rtd.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|                          | table blocks           | None<br><b>(Mandatory)</b> | These define the tables which are joined relationally to form a database "row". Each row corresponds with a datastream message.<br><br>Described fully below.                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>New in<br/>v6.1.5</b> | rowSafeWhere           | "1"                        | An integer-valued expression which can be used to indicate when a row is safe to send on the stream. This is typically used when we need to hold a row, and prevent it being sent on a stream before some asynchronous activity has been performed.<br><br>This expression is evaluated for every updated row. If it evaluates to a non-zero value, the row is passed out to each stream for transmission. If it evaluates to 0, the row is held, and no further updates are processed until the expression evaluates to a non-zero value for the held row. The expression will be re-evaluated for the held row every update poll period. |
| <b>New in<br/>v6.1.5</b> | rowSafeWarningSecs     | 5                          | If a row is held up by the <code>rowSafeWhere</code> expression, a warning is printed to the log file if the delay exceeds this value, in seconds.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |



**New in  
v6.0.0**

| Name                                    | Default         | Description                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>where</code>                      | "1"             | An integer-valued expression string that acts as a post-filter and is evaluated against the current database row. The <code>logon</code> alias can be used from within this expression and it stands for the logon message sds. Thus the additional parameters can be passed on logon to customise post-filtering – see section 2.5.7 |
| <code>extraEvent blocks</code>          | No extra events | Include this block to specify additional events you want to fire off from a single database row. – see section 2.5.8                                                                                                                                                                                                                  |
| <code>extraEvent/<br/>where</code>      | "1"             | An integer-valued expression string evaluated against the current database row. It decides if the event is to be generated.                                                                                                                                                                                                           |
| <code>extraEvent/<br/>XLator</code>     | DEFAULT         | Defines translator between database records and the extra event datastream messages. Can be <code>DEFAULT</code> , <code>EMBED</code> or <code>NONE</code> . Other values may be valid in applications linked against the stream server library – see below for further details.                                                      |
| <code>extraEvent/<br/>fieldExprs</code> | No expressions  | A list of expressions which are evaluated after the translator has been applied, in the context of the extra event output message and the current database row. Can be used to enhance a code-based translation, or build up a message through configuration alone.                                                                   |
| <code>fieldExprs</code>                 | No expressions  | A list of expressions which are evaluated after the translator has been applied, in the context of the output message and the current database row. Can be used to enhance a code-based translation, or build up a message through configuration alone.                                                                               |

|                      | Name                                              | Default                                              | Description                                                                                                                                                                                                                                                    |
|----------------------|---------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>New in v6.1.3</b> | <code>streamStatus block</code>                   | No recording of stream status                        | This block configures the optional recording of RDS status to an RTD table.<br><br>The block is optional, but if specified, a number of items are mandatory.                                                                                                   |
| <b>New in v6.1.3</b> | <code>streamStatus/table</code>                   | Mandatory if <code>streamStatus</code> block present | Specifies the RTD table to which RDS status is written.                                                                                                                                                                                                        |
| <b>New in v6.1.3</b> | <code>streamStatus/<br/>fieldExprs</code>         | Mandatory if <code>streamStatus</code> block present | List of expressions, parsed in the context of the above table and the stream's logon SDS, which should set enough fields in the table to allow a join to a unique record.                                                                                      |
|                      | <code>streamStatus/<br/>lastMsgRefField</code>    | Mandatory if <code>streamStatus</code> block present | Name of the field on the above table which is used to record the <code>DATASTREAM_REF</code> of the most recent message sent on this stream.<br><br>The field must be of type <code>dct_chararray</code> .                                                     |
| <b>New in v6.1.3</b> | <code>streamStatus/<br/>statusField</code>        | Mandatory if <code>streamStatus</code> block present | Name of the field on the above table which is used to record the status of this particular logon. This is set to D if the client in question is not logged on, and C if a logon is currently active.<br><br>This field must be of type <code>dct_char</code> . |
| <b>New in v6.1.3</b> | <code>streamStatus/<br/>startupUpdateWhere</code> | No records updated at startup                        | If specified, the stream server will scan the configured table at startup, using the specified expression to filter out rows which should be updated with <code>startupUpdateExprs</code> (below).                                                             |

|                       | Name                                              | Default                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------|---------------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>New in v6.1.3</b>  | <code>streamStatus/<br/>startupUpdateExprs</code> | No records updated at startup          | Specifies a list of expressions, evaluated at startup against every record matching <code>startupUpdateWhere</code> . Once these expressions have been applied, the record is written back to the RTD.<br><br>This could be used, for instance, to set all the statuses for a particular stream to "disconnected" when the process restarts.                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>New in v8.0.12</b> | <code>fixedFields block</code>                    | No fixed fields                        | Include this block to set one or more key fields from values supplied on the logon message sds.<br><br>Setting key fields allows the <code>StrSvr</code> to scan a sub-set of the records in the primary table when a client connects.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                       | <code>fixedFields/<br/>validateExpr</code>        | No validation of the logon message sds | An expression that can be used to validate the logon message sds.<br><br>This can be useful if the logon message sds is expected to contain non-standard fields, i.e. those not defined in the RDS protocol, the values of which are used to set key fields.<br><br>The logon message sds is made available to the expression as alias "logon".<br><br>The expression must return an empty string if the logon message sds is valid, or a non-empty string if the logon message sds is invalid.<br><br>If a non-empty string is returned, the client's logon attempt will be rejected. The non-empty string returned by the expression must contain a description of the error, as it will be used in the <code>NACK</code> returned to the client. |

| Name                                    | Default                                                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>fixedFields/<br/>setupExpr</code> | Mandatory of the <code>fixedFields</code> block is present | <p>An expression that can be used to set key fields on the database scanner's primary record from values supplied on the logon message sds.</p> <p>The logon message sds is made available to the expression as alias "logon".</p> <p>A blank primary record is also made available to the expression. The expression must set at least one key field on the primary record. Any key fields set by the expression must be on the database key identified by the <code>key</code> configuration item.</p> <p>Key fields set by the expression must be contiguous at the start of the key, so if the expression only sets one key field it must set the first field on the key, if the expression sets two key fields it must set the first two fields on the key, and so on.</p> |

### 2.5.1 Keying Your Stream

The `key` item specifies which key on the primary table will be used for scanning and for generating datastream references. This key should be sorted (so that `StrSvr` can scan down it in order), and it should either increase or decrease monotonically with time, i.e. every update which comes in should have a key value which is always either greater or less than the previous update. For example, a timestamp key satisfies this criterion (as the timestamp always increases), but a key on an internal trade identifier may not increase monotonically, as old trades may be amended.

A key may be used which is monotone decreasing as well as increasing. To use a decreasing key, set `reverseSort` to `YES`.

### 2.5.2 Update Regime

Versions of `StrSvr` prior to v6.0.0 had a complex regime for polling for updates, which involved matching new data with updates in order to guarantee only sending data which had been correctly journalled. If, for any reason, data could not be matched with updates, `StrSvr` would produce the following operator message:

```
30 JUN 1998,10:48:06,WAR,INF_STRSVR-0-Reliable stream lagging behind - check log file
```

Note that in v6.0.0 and later infrastructure releases, the update regime has been changed such that this can no longer occur.

### 2.5.3 Flow Control and Timeslicing

Rows are sent to clients in batches, to enable flow control. If the client does not specify a batch size when logging on, a default will be used. You can specify the default number of rows in a batch with the `defaultMaxCount` item.

In order to service multiple clients at once, `StrSvr` timeslices scans. A certain number of rows are scanned on behalf of each client before moving on to service a different request. You specify a number of rows to scan in the `rowsPerTimeslice` item.

### 2.5.4 Translating Records into Datastream Messages

The `XLator` item defines how rows are translated into messages for transmission to the client. It can take the value `DEFAULT`, in which case the default translation is used: this turns a database record into a SDS buffer with the same name as the record, and with the fields named within that SDS buffer, with a type appropriate to their database type. This default translation can be enhanced through expressions - see below for more details.

The `XLator` item can also take the value `NONE`, in which case no translation is applied, and the datastream message must be set up purely in configuration. The following sections show how to do this.

The `XLator` item can also take the value `embed`. This will embed in the returned SDS a repeating SDS of the records that are specified in the `Embed` block. For example, user data is specified in the `rdsTemplate` and the groups to which the user belongs could be returned in the repeating SDS array. - see Appendix A: The `EMBED` `XLator` for more information.

If the stream node library has been linked into another application and extra translators added, these can be named here. For example, the `TrdStr` process in FTS defines a specialised translator called `TRADE` which translates records from the FTS trade tables into the documented external representation.

If the default translator is in use, the following can also appear in your `rdsTemplate` block:

```

defaultXLator
{
    omitFields ( "NULL_MAP", "TIMESTAMP" )
}

```

This block specifies a list of fields in each record which the default translator will omit from its translation. Combined with the `fieldExprs` list (see below) this allows you to customise the default translation to your own requirements.

### 2.5.5 Enhancing Translation with Expressions

The `fieldExprs` item (which follows the `table` blocks described below) is a list of expressions which are evaluated after the translator has been applied. They are evaluated in the context of the scanned database row, so aliases of records in this row can be used to refer to particular fields. You can also write fields into the message to be sent with these expressions. Any item with no alias is assumed to refer to the outgoing message, and as with all SDS expressions, you should specify the type with which to address the data.

For example, the following configuration might be used in the FTS `TrdStr` process to add a supplementary field to the hard-coded conversion:

```

rdsTemplate
{
    ! Standard configuration, etc etc

    ! Use hard-coded trade model translator
    XLator TRADE

    ! Add an additional field
    fieldExprs ( "TRADE.CUSTOM.DATABASE_TIMESTAMP.s = TIMESTAMP.FULL" )
}

```

**Note:** See the *Fidessa Expression Evaluator Programmer's Guide* for full details of SDS and database record expressions, and for available expression functions and operators.

### 2.5.6 Configuring the Tables to Scan

The `table` blocks configure the tables to be scanned and the relational joins between them. Within these blocks, you can define:

- Filters for individual tables which are expression based.
- Aliases for individual records, allowing them to be uniquely referenced in subsequent expressions.

The exact configuration is described in the *Fidessa Expression Evaluator Programmer's Guide*.

In addition to the standard configuration, there are two embedded variables maintained by `StrSvr`, for use in your `where` expressions. These are `joinOk`, which is non-zero if

the join to the current record succeeded (i.e. at least one corresponding record was found) and `isUpdate`, which is non-zero if the current row was generated from an update to the database (as opposed to being recovered as part of an initial scan). `joinOk` is clearly only available in non-primary records, but `isUpdate` is available in all `where` expressions.

The following example shows the use of `joinOk` and `isUpdate` to filter a scan.

```
table
{
    name TRADE_SUMMARY
    alias ts
}
table
{
    name ALT_INSTRUMENT_ID
    join "ai.INSTRUMENT_ID = ts.INSTRUMENT_ID;
        ai.ALTERNATE_TYPE = 'RIC'"
    ! Only return a row when the underlying instrument has a RIC code,
    ! and only do so for an update
    where "joinOk && isUpdate"
}
```

### 2.5.7 Parameterised logon

**New in  
v6.0.0**

The `where` expression is introduced in order to allow for the post-filter parameterisation from reliable datastream clients. In addition to the standard aliases a new `logon` alias can also be used. This provides access to the complete message sent by client on logon.

The following example makes the stream server stream out only the rows time-stamped with the same date as the one of the logon datastream reference:

```
where "Mid$(logon.DATASTREAM_REF.s,0,8) == Mid$(t.TIMESTAMP,0,8)"
```

### 2.5.8 Extra events

**New in  
v6.0.0**

There is an option of enabling multiple events to be generated from a single database row by specifying them in the `extraEvent` blocks. For example, if only an amend is stored in the RTD, but a separate cancel and rebook are required on the stream.

Extra events are fired after the default, normally sent, message, and each of them has its own filter, translator and field expressions. These are configured in the same fashion as for the main event.

In order to guarantee recoverability the design of the event matching datastream reference has changed slightly. It now includes an extra delimiter and a sequence number that identifies event in a sequence for matching database row. Sequence number starts from 0 and increases for successive events accordingly.

Example configuration:

```

table
{
    name STRSVR_TEST
    alias t
}

extraEvent
{
    where "t.USER_ID > 1"
    XLator NONE
    fieldExprs ("SYSTEM.ITEM_NAME.s = t.USER_NAME",
                "SYSTEM.ITEM_VALUE.s = {string}t.USER_ID")
}

```

### 2.5.9 Worked Example

This is a more practical adaptation of the illustrative example above.

```

rdsTemplate
{
    name          TRADE_STREAM
    key           TRADE_SUMMARY_BY_AMENDED_DT
    XLator        NONE

    table
    {
        name TRADE_SUMMARY
        alias ts

        ! Illustrative "where" clause picks out only
        ! original version of principal trades
        where "ts.TRADE_BASIS=='PR' && VERSION==1"
    }

    table
    {
        name ALT_INSTRUMENT_ID
        alias ai
        join "ai.INSTRUMENT_ID = ts.INSTRUMENT_ID;
            ai.ALTERNATE_TYPE = 'RIC' "
        where "joinOk"
    }

    fieldExprs ( "TRADE.TRADE_SET_ID.s = ts.TRADE_SET_ID",
                  "TRADE.COUNTERPARTY.s = ts.COUNTERPARTY_ID",
                  "TRADE.INSTRUMENT.s = ai.INSTRUMENT_CODE",
                  "TRADE.PRICE.f64 = ts.TRADE_PRICE",
                  "TRADE.VOLUME.f64 = ts.VOLUME" )
}

```

The following tables show the format of the first three datastream messages received when logging on for the first time, with a `MAX_COUNT` of 3.

| Field Name | Format | Value |
|------------|--------|-------|
|------------|--------|-------|



| Field Name         | Format | Value                                             |
|--------------------|--------|---------------------------------------------------|
| MESSAGE_TYPE       | string | DS_MESSAGE                                        |
| DATASTREAM_REF     | string | "19980415 11:17:42.020000<br>+0100s 00000001LO 1" |
| MORE               | string | 1                                                 |
| TRADE.TRADE_SET_ID | string | 00000001LO                                        |
| TRADE.COUNTERPARTY | string | A                                                 |
| TRADE.INSTRUMENT   | string | ICI.L                                             |
| TRADE.PRICE        | F64    | 10.88                                             |
| TRADE.VOLUME       | F64    | 1200                                              |

| Field Name         | Format | Value                                             |
|--------------------|--------|---------------------------------------------------|
| MESSAGE_TYPE       | string | DS_MESSAGE                                        |
| DATASTREAM_REF     | string | "19980415 11:17:58.020000<br>+0100s 00000002LO 1" |
| MORE               | string | 1                                                 |
| TRADE.TRADE_SET_ID | string | 00000002LO                                        |
| TRADE.COUNTERPARTY | string | A                                                 |
| TRADE.INSTRUMENT   | string | ICI.L                                             |
| TRADE.PRICE        | F64    | 10.88                                             |
| TRADE.VOLUME       | F64    | 1500                                              |

| Field Name     | Format | Value                                             |
|----------------|--------|---------------------------------------------------|
| MESSAGE_TYPE   | string | DS_MESSAGE                                        |
| DATASTREAM_REF | string | "19980415 11:18:12.040000<br>+0100s 00000003LO 1" |
| MORE           | string | 1                                                 |

| Field Name         | Format | Value      |
|--------------------|--------|------------|
| TRADE.TRADE_SET_ID | string | 00000003LO |
| TRADE.COUNTERPARTY | string | A          |
| TRADE.INSTRUMENT   | string | ICI.L      |
| TRADE.PRICE        | F64    | 10.87      |
| TRADE.VOLUME       | F64    | 1249       |

## 2.6 Customising RDS

**New in  
v7.0.11**

An RDS may be customised to add additional tables to the scanner and to change the set of fields published on the stream. An RDS is customised using a `customiseRDSTemplate` block.

A `customiseRDSTemplate` block may contain the following items.

| Name                    | Default                    | Description                                                                                                                                                                                                                      |
|-------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>name</code>       | None<br><b>(Mandatory)</b> | Name of the RDS that is being customised.                                                                                                                                                                                        |
| <code>table</code>      | No additional tables.      | These blocks have the standard format for secondary tables in a database scanner and are used to add new tables to scan. See the Expression Evaluator Programmers guide for full details.                                        |
| <code>fieldExprs</code> | No additional expressions. | Extend the set of expressions evaluated to build messages for this template. These expressions have access to all of the tables from the database scanner including those added in this <code>customiseRDSTemplate</code> block. |
| <code>extraEvent</code> | No additional extra event. | Extend the set of extra events provided by this RDS. These blocks have the same format as <code>extraEvent</code> blocks in the <code>rdsTemplate</code> block.                                                                  |

### 2.6.1 Example

This shows customisation of the RDS from section 2.5.9. Here we want to add the field `CONSIDERATION_FACTOR` from the `INSTRUMENT` table.

```

customiseRDSTemplate
{
    name TRADE_STREAM

    table
    {
        name INSTRUMENT
        alias in
        join "in.INSTRUMENT_ID = ts.INSTRUMENT_ID"
    }

    fieldExprs ( "TRADE.CONSIDERATION_FACTOR.f64 = in.CONSIDERATION_FACTOR" )
}

```

## 2.7 Configuring Transaction Streams

To provide an outgoing stream of transactions, you must configure in a transaction template block. This block is formatted as follows:

```

txnTemplate
{
    sourceId      "'FTS'"
    key           SYSTEM_BY_TIMESTAMP
    xLator        DEFAULT
    generateSourceRef TRUE
    maxOutstanding 5

    txnService    "'TXN'"
    messageType   "'TXN_ADD'"
    userName      "'SPECIAL'"

    table
    {
        name SYSTEM
        where "Len$(ITEM_NAME) < 30"
    }

    fieldExprs ( )

    resultEvents
    {
        recordResultsToTable TRADE_STATUS
        recordWhere "Left$(request.ITEM_NAME,1)=='A'"
        fieldExprs ( "STATUS = If(reply.MESSAGE_TYPE.s == 'TXN_ACK',
                                'Y',
                                'N')")
        onNack      "If(reply.MESSAGE_TYPE.s == 'TXN_TXN_STATE',
                    Display('Recovered transaction ' +
                                request.SOURCE_REF.s +

```

```
        ' was NACKed\n'),
        Display('FTS NACKed transaction ' +
            request.MESSAGE_TYPE.s + ' ' +
            request.SOURCE_REF.s + ' ' +
            '\nError was: ' +
            reply.ERROR.ERROR_TEXT.s + '\n'))"
    }

    onLogon
    {
        tableName      LOG_EVENTS
        alias           le
        keyName         LOG_EVENTS_BY_USER_OA_TXN

        where          "userName=='TXN_USER'"

        fieldExprs ( "le.USER_NAME=userName",
                    "le.OA_SERVICE=oaService",
                    "le.RDS_NAME=rdsName",
                    "le.LOG_STATUS=logonFlag" )
    }
}
```

The `txnTemplate` block supports many items with the same names as the `rdsTemplate` block, and these have the same meaning in both blocks except as noted. The following additional items are also supported.

| Name                     | Default                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>messageType</code> | None                       | Expression which is evaluated every time a transaction is sent, in the context of the current database row. The result of the expression is added to the outgoing message as the <code>MESSAGE_TYPE</code> field. This allows you to configurably send different transaction types depending on underlying data.<br><br>If not specified, no <code>MESSAGE_TYPE</code> is added to the outgoing message, and this field should be added by your <code>fieldExprs</code> . |
| <code>sourceId</code>    | STRSVR                     | A simple expression (i.e. no context) which is evaluated every time a transaction is sent. Result added to outgoing message as <code>SOURCE_ID</code> .                                                                                                                                                                                                                                                                                                                   |
| <code>userName</code>    | None                       | A simple expression which is evaluated for every message in order to form the <code>USER_NAME</code> field in the outgoing transaction.<br><br>The value of the <code>USER_NAME</code> field may be modified by <code>xlators</code> and the <code>fieldExprs</code> block.                                                                                                                                                                                               |
| <code>txnService</code>  | None<br><b>(Mandatory)</b> | A simple expression which specifies the OpenAccess service to which this template sends transactions.                                                                                                                                                                                                                                                                                                                                                                     |

**New in  
v6.1.13**

| Name              | Default | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| initialSourceRef  | None    | <p>By default, if there is no resumption point for a particular stream (i.e. the transaction sink indicates there is no last <code>SOURCE_REF</code>), the stream will be barred and will not stream rows from the RTD; this is a safety mechanism, designed to avoid accidental full replay of streams. However, this can make it difficult to bootstrap a new stream.</p> <p>This optional configuration item defines an expression which will be evaluated if there is no last <code>SOURCE_REF</code> for this stream; the value returned will be used as if it were the initial <code>SOURCE_REF</code>.</p> <p>As most streams are keyed by timestamp, the expression <code>DctDateTimeToString(DctDateTimeNow())</code> should be sufficient to ensure a safe bootstrap for many streams.</p> |
| generateSourceRef | TRUE    | <p>If set to TRUE, <code>SOURCE_REF</code>s are generated, and transaction stream is resumed from last <code>SOURCE_REF</code> at start-up.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| maxOutstanding    | 1       | <p>Maximum number of transactions allowed to be outstanding with the transaction service at any one time.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

| Name                    | Default                           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sendUpdateLastSourceRef | TRUE                              | <p>From v5.0.0, as StrSvr starts, it will send a <code>TXN_UPDATE_LAST_SOURCE_REF</code> transaction to each transaction sink. This allows transaction sinks based on the v5 OpenAccess transaction framework to retain a limited number of <code>SOURCE_REF</code> records.</p> <p>If the transaction sink is not based on the v5 transaction framework, it is likely to NACK this transaction, which will cause a harmless warning in the StrSvr log file. To disable the sending of this transaction (and hence inhibit the warning) set this configuration item to <code>FALSE</code>.</p> |
| preSyncIdlePeriod       | 600                               | Number of seconds of inactivity after which transaction and result streams are synced up.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| fieldExprs              | No expressions                    | This is the same as for the <code>rdsTemplate</code> block, however, an embedded variable is available for use in the expressions. The variable is named "isRecovering", and it takes the value 1 when the template is recovering the states of the last <code>maxOutstanding</code> messages at the start of a session; otherwise (during normal operation) it takes the value 0. Note that this variable will only ever take the value 1 when source references are being used (see section 2.6.1).                                                                                          |
| onLogon                 | No recording of Log-on/off events | <p>This is the same as for the <code>rdsTemplate</code> block; however, for transaction streams, the <code>logonFlag</code> is set to "LOGGED_ON" when the StrSvr has requested the last source reference for recoverable streams or when the transaction server is available for non-recoverable streams.</p> <p>The <code>logonFlag</code> is set to "LOGGED_OFF" when the session has ended.</p>                                                                                                                                                                                            |

**New in  
v6.0.0**

| Name         | Default | Description                                                                                              |
|--------------|---------|----------------------------------------------------------------------------------------------------------|
| resultEvents | None    | Block which defines actions that occur when a transaction response is received. Described in full below. |

### 2.7.1 Source References

Source references can be generated by `StrSvr`, and if they are, the current position in the transaction stream will be determined by `StrSvr` using the `GET_LAST_SOURCE_REF` mechanism on start-up, as defined by the `OpenAccess` transaction protocol. You can turn on generation of source references by specifying `generateSourceRef` as `TRUE`.

In order to recover its last `SOURCE_REF`, the above configuration will generate the following message at start-up:

| Field Name   | Format | Value                   |
|--------------|--------|-------------------------|
| MESSAGE_TYPE | string | TXN_GET_LAST_SOURCE_REF |
| USER_NAME    | string | SPECIAL                 |
| SOURCE_ID    | string | FTS                     |

If no `SOURCE_REF` was then recovered, the following transaction might then be generated:

| Field Name        | Format | Value                           |
|-------------------|--------|---------------------------------|
| MESSAGE_TYPE      | string | TXN_ADD                         |
| USER_NAME         | string | SPECIAL                         |
| SOURCE_ID         | string | FTS                             |
| VALIDATE          | I32    | 0                               |
| IGNORE_WARNINGS   | I32    | 1                               |
| SOURCE_REF        | string | 19980415 10:00:58.958789 +0100s |
| SYSTEM.TIMESTAMP  | string | 19980415 10:00:58.958789 +0100s |
| SYSTEM.ITEM_NAME  | string | MINDER_OPERATOR_ALERT           |
| SYSTEM.ITEM_VALUE | string | OK                              |

### 2.7.2 Asynchronous Behaviour

By default, the behaviour of the transaction stream is synchronous, i.e. a transaction is generated and sent and the reply is waited for before generating another transaction. This behaviour can be modified by setting `maxOutstanding` to an integer greater than 1. If this is done, a number of transactions are allowed to be outstanding with the



transaction server before waiting for any responses, so, if 5 is configured in this item, `StrSvr` will generate 5 transactions and send them to the transaction service before waiting for the replies. When the first reply is received, another transaction will be generated, ensuring that this number of transactions are always outstanding. Additionally, on start-up, if `StrSvr` recovers the position in the transaction stream, it will also recover the statuses of this number of preceding transactions. For example, if `maxOutstanding` is set to 5, and `StrSvr` starts up and finds that the tenth transaction in a stream was the last one to be processed, it will recover the statuses of the tenth, ninth, eighth, seventh and sixth transactions in the stream.

**Note:** Asynchronous behaviour, using large outstanding counts, can result in greatly increased performance.

### 2.7.3 Synchronisation

To avoid time consuming RTD forward scans in case of rarely used streams, transaction and result streams are synced up. The synchronisation is according to the latest record in the real time database, just after its activity has quiesced. In order to enforce this, an expiry of the idle period, set by `preSyncIdlePeriod` configuration item, triggers the synchronisation messages to the corresponding transaction sinks.

**Note:** Normally, this item is not specified.

### 2.7.4 Result Events

The next block of interest is the `resultEvents` block. This block specifies what `StrSvr` does when the transaction service replies to a transaction. Configurably, when a reply is received, `StrSvr` can:

- Update a local database table
- Send another transaction into a local or remote transaction service (a “status” transaction)
- If the transaction was `NACKed`, execute a pre-defined expression.

All the actions are configured in the `resultEvents` block, as follows:

| Name                              | Default | Description                                                                                                                                                                                                                                      |
|-----------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>recordResultsToTable</code> | None    | Name of a table to write a result record to. If this item is specified, a result record is generated for every transaction response received, and every transaction status recovered on startup, subject to the <code>recordWhere</code> clause. |

| Name                     | Default  | Description                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>recordWhere</code> | non-zero | Expression which filters results transactions. When a transaction service replies to a transaction, and this expression evaluates to a non-zero value, a record is created in the table specified by <code>recordResultsToTable</code> . By default, all results create a record.<br><br>Evaluated in the context of both the transaction result and reply message (aliased as <code>request</code> and <code>reply</code> ). |
| <code>fieldExprs</code>  | None     | List of expressions which are used to fill in the record to be created in <code>recordResultsToTable</code> .<br><br>Evaluated in the context of both the transaction result and reply message (aliased as <code>request</code> and <code>reply</code> ). The record which is being created is aliased as <code>record</code> .                                                                                               |
| <code>onNack</code>      | None     | List of expressions to evaluate if a transaction is <code>NACKed</code> , or a recovered transaction had a <code>NACKed</code> status.<br><br>Evaluated in the context of both the transaction result and reply message (aliased as <code>request</code> and <code>reply</code> ).                                                                                                                                            |
| <code>resultTxn</code>   | None     | Defines a transaction which is sent when a reply to the original transaction is received.                                                                                                                                                                                                                                                                                                                                     |

An example of the `resultTxn` block follows:

```
resultTxn
{
    txnService    "FTS_ERR"
    reportWhere   "reply.MESSAGE_TYPE.s = 'TXN_NACK'"

    messageType  "'TXN_REPLACE'"
    userName     "'SPECIAL'"
    sourceId     "'SOES_AUTOEXEC'"

    fieldExprs ( "txn.TRADE_STATUS.SOURCE_REF.s =
                  request.SOURCE_REF.s")
}
```

The following items are valid in this block.

| Name        | Default                    | Description                                                                                                                                                                                    |
|-------------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| txnService  | None<br><b>(Mandatory)</b> | Expression which evaluates to the name of the service into which to play the transaction.<br>Evaluated once, with no additional context.                                                       |
| reportWhere | non-zero                   | Evaluated for every reply received. If it evaluates to a non-zero value, a result transaction will be generated. By default, all replies generate a result transaction.                        |
| messageType | None<br><b>(Mandatory)</b> | Expression which evaluates to MESSAGE_TYPE for the result transaction.                                                                                                                         |
| userName    | None<br><b>(Mandatory)</b> | Expression which evaluates to USER_NAME for the result transaction.                                                                                                                            |
| sourceId    | None<br><b>(Mandatory)</b> | Expression which evaluates to SOURCE_ID for the result transaction.                                                                                                                            |
| fieldExprs  | None                       | List of expressions which are evaluated to form the result transaction.<br><br>An additional alias, <code>txn</code> , is available in this list, referring to the result transaction message. |

Where not explicitly stated otherwise, the “request” and “reply” aliases are available in all these expressions.

The above example would therefore produce the following transaction message, if an original transaction with SOURCE\_REF TXN\_100 was NACKED.

| Field Name              | Format | Value         |
|-------------------------|--------|---------------|
| MESSAGE_TYPE            | string | TXN_REPLACE   |
| USER_NAME               | string | SPECIAL       |
| SOURCE_ID               | string | SOES_AUTOEXEC |
| VALIDATE                | I32    | 0             |
| IGNORE_WARNINGS         | I32    | 1             |
| TRADE_STATUS.SOURCE_REF | string | TXN_100       |

## 2.8 Customising Transaction Streams

*New in*  
**v7.0.11**

A transaction stream may be customised to add additional tables to the scanner and to change the set of fields published on the stream. A transaction stream is customised using a `customiseTxnTemplate` block.

A `customiseTxnTemplate` block may contain the following items.

| Name                    | Default                                          | Description                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceId</code>   | None<br><b>(Mandatory)</b>                       | This expression must produce the same result value as the <code>sourceId</code> expression from the transaction template being customised.                                                                                                                                                                                                      |
| <code>txnService</code> | None<br><b>(Mandatory)</b>                       | This expression must produce the same value as the <code>txnService</code> expression from the transaction template being customised.                                                                                                                                                                                                           |
| <code>table</code>      | No additional tables.                            | These blocks have the standard format for secondary tables in a database scanner and are used to add new tables to scan. See the Expression Evaluator Programmers guide for full details.                                                                                                                                                       |
| <code>fieldExprs</code> | No additional expressions.                       | Extends the set of expressions evaluated to build messages for this template. These expressions have access to all of the tables from the database scanner including those added in this <code>customiseTxnTemplate</code> block.                                                                                                               |
| <code>resultTxn</code>  | No additions to the original result transaction. | If the original template has no <code>resultTxn</code> block, this must be a complete <code>resultTxn</code> block.<br><br>If the original template has a <code>resultTxn</code> block, this block can contain a <code>fieldExprs</code> block, and these expressions are evaluated as for the <code>fieldExprs</code> from the original block. |

## 2.9 Data Fresh Where

The `dataFreshWhere` block allows `StrSvr` to configurably change its status depending on values in a database table. This can be used to force a failover between broadcast systems, for example, when a link to an external data provider fails.

When data is deemed to be fresh, StrSvr's OpenAccess service is up, and reliable datastreams can be requested by client applications. Additionally, transaction streams are active. When data is deemed to be stale, StrSvr's OpenAccess service is down, and hence reliable datastreams are not available. Also, transaction streams are stopped.

The `dataFreshWhere` block is configured as follows:

```
dataFreshWhere
{
    table SESSION_STATUS

    dataFreshWhere "SERVICE_CODE=='BDG044' && SESSION_STATUS=='U'"
    dataStaleWhere "SERVICE_CODE=='BDG044' && SESSION_STATUS=='D'"
}
```

When the `DATABASE_AVAILABLE` state is received, the configured table is scanned, and the `dataFreshWhere` expression is evaluated for each record. The data is deemed to be fresh if the expression evaluates to a non-zero value for any record.

After the initial scan, StrSvr listens for updates to the specified table, and does the following:

- If the data is currently fresh, the `dataStaleWhere` expression is evaluated for each update. If the expression evaluates to a non-zero value, the data is deemed to be stale.
- If the data is currently stale, the `dataFreshWhere` expression is evaluated for each update. If the expression evaluates to a non-zero value, the data is deemed to be fresh.

This arrangement of expressions allows for cases where the data fresh and data stale conditions are not mutually exclusive; e.g. in the example, the `SESSION_STATUS` flag is tri-state.

The following table summarises the configuration options for the `dataFreshWhere` block.

| Name                        | Default                    | Description                                                                                                                                                                                                                                                        |
|-----------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>table</code>          | None<br><b>(Mandatory)</b> | Defines the table to scan. This can also be configured to hold table blocks as used by the database scanner.                                                                                                                                                       |
| <code>dataFreshWhere</code> | None<br><b>(Mandatory)</b> | Expression which should return an integer value. Data is initially fresh if a non-zero value is returned for any record in the configured table. If data is stale, it becomes fresh when an update occurs for which this expression evaluates to a non-zero value. |

| Name           | Default                      | Description                                                                                                       |
|----------------|------------------------------|-------------------------------------------------------------------------------------------------------------------|
| dataStaleWhere | 0 (data never becomes stale) | When data is fresh, it becomes stale if an update occurs for which this expression evaluates to a non-zero value. |

The table entry can be configured as table blocks for use by the database scanner. The previous example could be configured as follows:

```
dataFreshWhere
{
  table
  {
    name SESSION_STATUS
    alias s
  }

  dataFreshWhere "s.SERVICE_CODE=='BDG044' && s.SESSION_STATUS=='U'"
  dataStaleWhere "s.SERVICE_CODE=='BDG044' && s.SESSION_STATUS=='D'"
}
```

## 2.10 Special Note: Guaranteed Ticks

**Updated in  
v8.0.0**

From v8.0.0, `StrSvr` insists that its primary table has a guaranteed tick update queue. This allows it to very simply ensure that all updates have been journalled before being sent on a stream. This replaces earlier, more complex mechanisms and increases performance.

## 3. Process operation

### 3.1 Command Line Options

The command line options supported by `StrSvr` are as follows:

|                                    |                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------|
| <code>-c configuration file</code> | Specify an alternative configuration file. The default will be <code>StrSvr.cfg</code> |
|------------------------------------|----------------------------------------------------------------------------------------|

|                               |                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-allowFullReplay</code> | From infrastructure v6.0.0, <code>StrSvr</code> no longer performs the full rewind if sent an empty string message reference. This applies for both transaction and reliable data streams. In order to support the cold start of external systems, <code>-allowFullReplay</code> needs to be specified for the stream server process, thus allowing for full replays. |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**New in  
v6.0.0**

## 4. Sample Configuration

The following configuration file is a modified version of that used to send executions from EMS into FTS. For clarity, and to provide a real-world example, many optional or unnecessary items are not shown here.

The example is not complete: the `fieldExprs` block do not contain the full complement of fields necessary to send trades, or failed trade reports, into FTS. Extra fields have been omitted for the sake of clarity.

```
!
!   SCCS Identifier      : Version %I% Title %M%
!   Title               : Stream Server configuration file
!   System              : Fidessa
!   Sub-System          : EMS to FTS interface example
!   Version             : 1.0
!   Copyright           : (c) royalblue technologies plc
!   Date                : 11th March 1998
!   Technical Reviewer  :
!
!   Modification History
!
!   Version             : 1.0
!   Date                : 11th March 1998
!   CRF                 :
!   Tech, Reviewer      :
!   Changes             : New File
!
!   Version             :
!   Date                :
!   CRF                 :
!   Tech, Reviewer      :
!   Changes             :
!
StrSvr
{
    general
    {
        processName      EMS_FTSSVR
        streamServiceName EMS_FTSSVR
    }

    expression
    {
```



```

! preExpressions used to set up global variables
preExpression ( "automaticExecution##s = 'AT';
                 dealingAsPrincipal##s = '';
                 dealingAsRiskless##s = 'RP';
                 dealingAsAgent##s = '';
                 counter##i = 0" )
}

linkMgr
{
    ipcName          EMS_FTSSVR

    #include
    {
        fileName     "ClientSpecific.cfg"
        configPath    "/OpenAccess"
    }
}

streamNode
{
    ! Use defaults for the general block

    txnTemplate
    {
        sourceId      "'EMS'"
        key            EXECUTION_BY_TIMESTAMP
        generateSourceRef TRUE
        XLator         NONE
        maxOutstanding 5

        txnService     "'FTS_OAT'"
        messageType    "'TXN_PRINCIPAL_ENTRY'"
        userName       "'EMS'"

        table
        {
            name ORDER_EXECUTIONS
            alias oe
            where "Len$(ORDER_ID) > 0"
        }
        table
        {
            name ACTIVE_ORDERS
            alias as
            join "as.ORDER_ID = oe.ORDER_ID"
            where "Len$(as.INT_INSTRUMENT_ID) > 0"
        }
        table
        {
            name INSTRUMENT
            alias inst
            join "inst.INT_INSTRUMENT_ID = as.INT_INSTRUMENT_ID"
            where "Len$(inst.EPIC_CODE) > 0"
        }
    }
}

```

```

    }
    table
    {
        name PARTICIPANT
        alias part
        join "part.COUNTERPARTY_ID = oe.COUNTERPARTY_ID"
    }

    fieldExprs
    (
        ! Update notification banner
        !
        "text#s='ORDER EXECUTION SERVICE - Checking execution';
        LogTrace$(text#s);
        Display('\tINFORMATION - Order '+as.ORDER_ID);",

        ! Increment the counter, this is used to ensure
        ! that multiple executions with the same trade code
        ! generate unique contributor references
        "value#s = Right$(oe.TRADE_TIMESTAMP,6);
        counter##i = {int}value#s % 1000;",

        ! Message header fields
        !
        "MF_VERSION.i32 = 1",
        "LOCATION.s = 'LO'",
        "TIMEZONE.s = 'GB'",

        ! Trader on execution is the order originator
        !
        "USER_NAME.s = as.OWNER",

        ! Set the trade ID, counter is used to ensure that
        ! multiple executions with the same trade code
        ! generate unique contributor references
        "TRADE_ID.s =
oe.EXCHANGE_TRADE_CODE+oe.TRADE_TYPE+(string)counter##i+'-'+oe.TRADE_DATE",

        ! Trade fields
        !
        "TRADE_ID_TYPE.s = 'EMS'",
        "TRADE.BUY_SELL.b = ByteAt(oe.TRADE_TYPE,0)",
        "TRADE.VOLUME.f64 = oe.TRADE_VOLUME",
        "TRADE.INSTRUMENT_ID.s = inst.INT_INSTRUMENT_ID",
        "TRADE.INSTRUMENT_ID_TYPE.s = 'LMIL'",
        "TRADE.COUNTERPARTY_ID.s = oe.COUNTERPARTY_ID",
        "TRADE.COUNTERPARTY_ID_TYPE.s = 'BIC'",
        "TRADE.NET_PRICE.f64 = oe.TRADE_PRICE",
        "TRADE.TRADE_CURRENCY.s = inst.CURRENCY",
        "TRADE.TRADE_DATE.s = oe.TRADE_DATE",
        "TRADE.TRADE_TIME.s = oe.TRADE_TIME",
        "TRADE.SALESMAN.s = as.SALESMAN"
    )

```

```
resultEvents
{
    ! Record NACKS by sending a transaction to FTS to
    ! record the details
    resultTxn
    {
        txnService    "'FTS_OAT'"
        reportWhere    "reply.MESSAGE_TYPE.s == 'TXN_NACK'"
        messageType    "'TXN_ADD'"
        userName        "'SPECIAL'"
        sourceId        "'EMS'"

        fieldExprs
        (
            "text#s='EXECUTION BOOKING FAILURE - '+request.TRADE_ID.s;
            LogTrace$(text#s);
            Display('\tREASON - '+reply.ERROR.ERROR_TEXT.s+'\n')",

            "txn.FAILED_TRADES.CONTRIBUTOR_ID.s = 'EMS'",
            "txn.FAILED_TRADES.CONTRIBUTOR_REF.s = request.TRADE_ID.s",

            !
            ! Etc, etc..  add all other fields
            !

            "Display('\tSENT - Failed trade report to FTS\n')"
        )
    }
}
}
```

## 5. Appendix A: The EMBED XLator

When the XLator item has the value `embed`, a 'one to many' relationship is applied. This turns the many matching database records into a SDS array.

The `rdsTemplate` block when making use of the EMBED XLator has the following format.

```
rdsTemplate
{
    name          TRADE_STREAM
    key           INSTRUMENT_BY_ID
    XLator        EMBED

    table
    {
        name  INSTRUMENT
        alias iid

    }

    ! Optional block describing the behaviour of the EMBED XLator
    embedXLator
    {
        xlatAllFields    TRUE
    }

    Embed
    {
        sdsName EXCHANGE

        join "ex.EXCHANGE_ID = iid.EXCHANGE_ID"

        table
        {
            name  EXCHANGE
            alias  ex
        }

        fieldExprs (
            "EX_TIMESTAMP.s"           = ex.TIMESTAMP",
            "EX_EXCHANGE_ID.s"         = ex.EXCHANGE_ID",
            "EX_DESCRIPTION.s"         = ex.DESCRPTION",
            "EX_COUNTRY_CODE.s"        = ex.COUNTRY_CODE"
        )
    }
}
```

```

Embed
{
    sdsName ALT_INSTRUMENT_ID

    join "aid.INSTRUMENT_ID = iid.INSTRUMENT_ID"

    table
    {
        name ALT_INSTRUMENT_ID
        alias aid
    }

    fieldExprs (
        "ALT_TIMESTAMP.s      = aid.TIMESTAMP",
        "ALT_INSTRUMENT_ID.s  = aid.INSTRUMENT_ID",
        "ALT_ALTERNATE_TYPE.s = aid.ALTERNATE_TYPE",
        "ORIG_STATUS.s = orig_iid.STATUS"
    )
}

fieldExprs (
    "INST_INSTRUMENT_ID.s = iid.INSTRUMENT_ID",
    "INST_INSTRUMENT_TYPE.s = iid.INSTRUMENT_TYPE",
    "INST_STATUS.s = iid.STATUS",
    "INST_DESCRIPTION.s = iid.DESCRPTION"
)
}

```

All the items relating to the `EMBED` XLator are defined below.

| Name                          | Default                    | Description                                                                                                                                                                                                                                                         |
|-------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| embedXLator/<br>xlatAllFields | FALSE                      | If set to <code>TRUE</code> , the XLator will pre-populate each output SDS with a default representation of the scanned row. In addition, the primary record of the rows joined to by each <code>embed</code> block will be used to pre-populate each embedded SDS. |
| embed/join                    | None<br><b>(Mandatory)</b> | Statement to be used to join from the tables from the <code>rdsTemplate</code> block to a table in the <code>Embed</code> block.                                                                                                                                    |
| embed/sdsName                 | None<br><b>(Mandatory)</b> | Name of the repeating SDS that will be added.                                                                                                                                                                                                                       |

Updated in  
v7.0.4

| Name             | Default                    | Description                                                                                                                                                  |
|------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| embed/fieldExprs | None<br><b>(Mandatory)</b> | List of expressions used to add fields to the embed sds. Fields in the original scanned row can be accessed by prefacing the alias with <code>orig_</code> . |

Given the previous configuration, the output could be as follows.

```
MESSAGE_TYPE.s = "DS_MESSAGE"
DATASTREAM_REF.s = "103518"
MORE.i32 = 1
  EXCHANGE[0].EX_TIMESTAMP.s = "20000503164226220123"
  EXCHANGE[0].EX_EXCHANGE_ID.s = "LSE"
  EXCHANGE[0].EX_DESCRIPTION.s = "London Stock Exchange"
  EXCHANGE[0].EX_COUNTRY_CODE.s = "GB"
  ALT_INSTRUMENT_ID[0].ALT_TIMESTAMP.s = "20000616162033219255"
  ALT_INSTRUMENT_ID[0].ALT_INSTRUMENT_ID.s = "103518"
  ALT_INSTRUMENT_ID[0].ALT_ALTERNATE_TYPE.s = "EPIC"
  ALT_INSTRUMENT_ID[0].ORIG_STATUS.s = "A"
  ALT_INSTRUMENT_ID[1].ALT_TIMESTAMP.s = "20000616162053849437"
  ALT_INSTRUMENT_ID[1].ALT_INSTRUMENT_ID.s = "103518"
  ALT_INSTRUMENT_ID[1].ALT_ALTERNATE_TYPE.s = "ISIN"
  ALT_INSTRUMENT_ID[1].ORIG_STATUS.s = "A"
INST_INSTRUMENT_ID.s = "103518"
INST_INSTRUMENT_TYPE.s = "EQ"
INST_STATUS.s = "A"
INST_DESCRIPTION.s = "Reflec Ord 5P"
```

## 6. Appendix B: OpenAccess Statistics

The stream server holds the following OpenAccess statistics:

### 6.1 Class <SERVICE\_NAME>.RDS

#### 6.1.1 Statistics

From version 5.1.2 two sets of statistics are available in the `DATASTREAM` SDS:

Statistics since the start of the process

- Statistics since the last reset

See section 6.1.3 for details on how to reset these fields.

| Field Name | Format | Notes                                                                      |
|------------|--------|----------------------------------------------------------------------------|
| DATASTREAM | REPSDS | SDS containing statistics for each datastream. May be repeated many times. |

Format for the datastream SDS:

| Field Name        | Format | Notes                                                    |
|-------------------|--------|----------------------------------------------------------|
| NAME              | string | Datastream name.                                         |
| PRIVILEGE_REJECTS | I32    | Number of rejections due to insufficient user privilege. |
| LOGONS            | I32    | Number of times datastream has logged on                 |
| TOTAL_MESSAGES    | I32    | Total number of messages sent since service started      |
| MESSAGES          | I32    | Number of messages sent for current session              |
| LAST_REF          | string | Last datastream reference sent.                          |
| LAST_MESSAGE_TIME | string | Time of last sent message in format HH:MM:SS             |
| LAST_LOGON_REF    | string | Datastream reference of last logon                       |
| LAST_LOGON_TIME   | string | Time of last datastream logon in format HH:MM:SS         |
| LAST_LOGOFF_REF   | string | Datastream reference of last logoff                      |

| Field Name               | Format | Notes                                                                                                 |
|--------------------------|--------|-------------------------------------------------------------------------------------------------------|
| LAST_LOGOFF_TIME         | string | Time of last datastream logoff in format HH:MM:SS                                                     |
| MIN_LOGON_DURATION       | F64    | Minimum number of seconds logged on (after successful logon)                                          |
| AVG_LOGON_DURATION       | F64    | Average number of seconds logged on (after successful logon)                                          |
| MAX_LOGON_DURATION       | F64    | Maximum number of seconds logged on (after successful logon)                                          |
| RESET_PRIVILEGE_REJECTS  | I32    | Number of rejections due to insufficient user privilege. , since the last reset of the statistics     |
| RESET_TOTAL_MESSAGES     | I32    | Total number of messages sent since service started, since the last reset of the statistics           |
| RESET_MESSAGES           | I32    | Number of messages sent for current session, since the last reset of the statistics                   |
| RESET_MIN_LOGON_DURATION | I32    | Minimum number of seconds logged on (after successful logon) , since the last reset of the statistics |
| RESET_AVG_LOGON_DURATION | I32    | Average number of seconds logged on (after successful logon) , since the last reset of the statistics |
| RESET_MAX_LOGIN_DURATION | I32    | Maximum number of seconds logged on (after successful logon), since the last reset of the statistics  |
| LAST_RTD_REF             | string | Datastream reference of last entry in the RTD.                                                        |

### 6.1.2 Counters

None.



### 6.1.3 Attributes

| Attribute name | Format | Read/Write | Notes                                                                                      |
|----------------|--------|------------|--------------------------------------------------------------------------------------------|
| COMMAND        | string | write only | Set to:<br><br>RESET_STATS to reset the<br>reset_xxx statistics to their default<br>values |

## 6.2 Class <SERVICE\_NAME>.TXN

### 6.2.1 Statistics

| Field Name | Format | Notes                                                                         |
|------------|--------|-------------------------------------------------------------------------------|
| DATASTREAM | REPSDS | SDS containing statistics for each<br>datastream. May be repeated many times. |

Format for the datastream SDS:

| Field Name | Format | Notes                                           |
|------------|--------|-------------------------------------------------|
| NAME       | string | Datastream name.                                |
| MESSAGES   | I32    | Number of messages sent for current<br>session. |
| LAST_REF   | string | Source reference of the last sent transaction.  |

### 6.2.2 Counters

### 6.2.3 Attributes

## 6.3 Class <SERVICE\_NAME>.INFO

### 6.3.1 Statistics

None

### 6.3.2 Counters

None

### 6.3.3 Attributes

| Attribute name | Read/Write | Notes                                                                                                                                                                                                                                                          |
|----------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HEALTH         | read only  | 0 - either no transaction streams are defined, or all transaction streams are up and synchronised<br>1 - a transaction stream is not connected / synchronised                                                                                                  |
| HEALTH_TXT     | read only  | Can contain two possible values depending on the health. <ul style="list-style-type: none"><li>• "All transaction streams are up and synchronised."</li><li>• "A transaction stream is not connected/synchronised."</li></ul>                                  |
| HEALTH_SRV     | read only  | Can contain two possible values depending on the health. <ul style="list-style-type: none"><li>• An empty string</li><li>• A list of transaction streams that are not connected/synchronised. The source and the service are listed for each stream.</li></ul> |

## 7. Appendix C: Pitfalls

### 7.1 Global Variable Use

Sometimes, global variables are set in database scanner expressions to avoid repeating configuration in the field expressions.

There are some issues regarding global variable use that require a special treatment.

```
table
{
    name ORDER
    where "globalVar##s=FIELD1; 1"
}
fieldExprs("FIELD2.s=globalVar##s")
```

Under heavy system load the value of the global variable set in the database scanner configuration can change before the field expressions get evaluated. Thus we end up using an overwritten value of `globalVar`.

In order to get round this a new technique was introduced. It uses a global property-value map where one can store, retrieve, and delete named property values using the appropriate expression evaluator functions (`MapAdd`, `MapGet`, and `MapDelete`).

The problem is solved by using some of the key field values to form a unique property name that will be stored in the global map along with its value.

Previous example rewritten using global map functions:

```
table
{
    name ORDER
    where "MapAdd('globalVar'+ ORDER_ID, FIELD1)"
}
fieldExprs("FIELD2.s=MapGet('globalVar'+ ORDER_ID)")
```

Values are now safely stored in the global map and they can be retrieved later.

### 7.2 Backward Joins

There is no support for backward joins within the stream server. In software versions prior to v6.1.3, they cause no errors but are silently ignored.

**New in v6.1.3** From v6.1.3 onwards, the stream server validates that no `backwardJoins` are present in the configuration.

## 7.3 Update Accumulation

After the initial scan has finished, the stream server may react to updates on relevant rows, configured up within database scanner blocks. That happens if their key changes in a way that it becomes “top of the list” and then an appropriate message is built and sent off against the configured stream. Thus many updates to a single row are published on a stream.

When performing a recovery on a stream, the stream server is only able to recover the last message on the stream originating from an update to a particular row. This means that the history information concerning consecutive updates on a row **is not** recovered and **should not** be expected. Inserted rows are fully recovered with the presumption they have not been deleted since they were last streamed out.

## 7.4 Stream Resumption

*New in  
v6.1.13*

v6.x.x stream servers will refuse to resume a transaction stream for which there is no last `SOURCE_REF`. This is a safety feature, designed to avoid the accidental replay of entire streams, but it can make it difficult to roll out new streams.

From v6.1.13, `StrSvr` supports an expression which can be specified in configuration, which allows an initial `SOURCE_REF` to be defaulted when none is present. This allows `StrSvr` to resume a stream from a safe point.

As most streams are keyed using database timestamps, a sensible resumption point is the current time; this ensures that historic data is never accidentally re-sent. The following configuration implements this:

```
txnTemplate
{
    ! ... etc ...
    initialSourceRef "DctDateTimeToString(DctDateTimeNow())"
}
```

## 8. Appendix D: The SOURCE\_KEY field

### **New in v6.1.3**

If two stream servers were to use the same `SOURCE_ID` for sending transactions to the same transaction sink, recovery problems would ensue. From v6.1.3 onwards, a degree of protection has been added to the stream server, such that only one process can use a particular `SOURCE_ID`.

This is implemented using the new `SOURCE_KEY` field, which is sent on every transaction. This field contains enough information to uniquely identify the sending process - primary and standby hostnames, runtime account name, and process name. When the stream server initially synchronises, the transaction sink records the `SOURCE_KEY` value; all subsequent transactions from the same `SOURCE_ID` must specify the same `SOURCE_KEY`. Thus if another process uses the same `SOURCE_ID`, the transactions would be rejected, avoiding recovery problems<sup>1</sup>.

This of course relies on the `SOURCE_KEY` being supported by the target transaction sink; this support has been added in v6.1.3. `SOURCE_KEY` values are stored in the `SOURCE` table.

### 8.1 Backward Compatibility

Because `SOURCE_KEY` is an additional, header-level field in each transaction, it could result in `NACKs` from existing transaction handlers. Compatibility issues are handled as follows:

#### 8.1.1 Post-v6.1.3 StrSvr, pre-v6.1.3 transaction processor

This case is handled automatically.

During the initial recovery sequence, the stream server spots that the transaction processor does not support the new field, and does not send it on subsequent transactions.

As a fail-safe, it is possible to disable the sending of the `SOURCE_KEY` field manually, by specifying the following configuration item:

---

<sup>1</sup> In fact, the initial synchronisation sequence would fail; the stream server recognises the reason for failure, and aborts, logging an appropriate message.

```
StrSvr
{
    ...

    streamNode
    {
        general
        {
            ...
            sendSourceKey FALSE
        }
        ...
    }
}
```

### 8.1.2 Post-v6.1.3 transaction processor, pre-v6.1.3 StrSvr

This case is handled automatically.

If the `SOURCE_KEY` is not specified on transactions, the transaction framework processes them normally. The only exception to this is, of course, if a new `StrSvr` has already used the appropriate `SOURCE_ID`, and has set the `SOURCE_KEY` value; in this case, transactions with no `SOURCE_KEY` will be rejected.



**For more information  
about Fidessa solutions**

[fidessa.com](https://fidessa.com)